ITAI 1378 | Module 09

# AI Agents for Computer Vision

What Becomes Possible When an Agent Can See


Student Reading Booklet

Companion to the Module 09 Slide Deck

**Author: Patricia McManus**

Original drafting collaborator: Claude (Anthropic)
Revised and technically reviewed with OpenAI ChatGPT, July 2026

# Table of Contents

## References and Further Reading

# How to Use This Booklet

This booklet is your reading companion for Module 09. Where Module 08 gave you the building blocks of modern computer vision, Module 09 wires them together into something new. An AI agent is a system that perceives, reasons, acts, and remembers. When that system can see, the range of work it can do expands enormously. This booklet walks you through what that means in plain English.

Two ground rules. First, there is no math in the body of this booklet. Every concept that has a mathematical foundation is explained in plain English. If you want to dig deeper later, the References section at the end points you to the right textbooks and papers.

Second, this booklet stays focused on computer vision. ITAI 2376 covers AI agents in depth across three full modules. We will name those topics where they come up so you know what to expect if you continue, but we will not duplicate that work here. What you get in this booklet is the CV slice.

## Learning Objectives

- Distinguish a computer vision model, a fixed pipeline, an automated system, and an AI agent.
- Identify perception, reasoning, action, observation, and memory in a vision-capable agent.
- Explain how specialized CV tools and multimodal models can work together.
- Describe ReAct as one influential agent pattern without assuming that every agent uses it.
- Select an appropriate architecture based on accuracy, latency, cost, privacy, and risk.
- Design and evaluate a small vision-capable agent with appropriate human oversight.

## Model, Pipeline, or Agent?

| Category | What it does | Acts? | Memory? | Example |
|---|---|---|---|---|
| CV model | Makes a prediction from visual input. | No | No | YOLO detects a person. |
| VLM | Interprets visual and language input. | Not by itself | Not by itself | Describes a possible safety concern. |
| CV pipeline | Runs a fixed sequence of processing steps. | Fixed actions | Optional | Detect, classify, and save a result. |
| CV agent | Selects tools or actions while pursuing a goal. | Yes | Often | Checks a scene, reviews history, and alerts a supervisor. |

## The Six Callout Boxes

Throughout the booklet you will see colored callout boxes. Each one has a job:

**KEY TAKEAWAY | Green callouts**

Summarize the one thing you must remember from a section. If you only have five minutes before class, read the green boxes.

**INDUSTRY PRACTICE | Teal callouts**

Show how this concept appears in real jobs. They are the bridge from theory to paycheck.

**DEEPER LOOK | Indigo callouts**

Go one layer deeper for the curious student. Optional. Skipping them will not hurt your grade, but reading them will help you on interviews.

**MISCONCEPTION ALERT | Red callouts**

Call out a common confusion. If you find yourself nodding at the wrong idea, the red box is your warning sign.

**THINK ABOUT IT | Amber callouts**

Pose a question to sit with. There is usually no single right answer.

**KNOWLEDGE CHECK | Blue callouts**

Short self quizzes. Answer them on paper before you move on.

# Part 01 | What Is an AI Agent?

Just enough foundation to follow the rest of the module

Before we look at how computer vision plugs into AI agents, we need a shared mental model of what an agent actually is. This is the part of the module that students from many different backgrounds find easiest to skim, because the word agent sounds like something complicated. The truth is closer to the opposite. The core idea is small. The implications are huge.

Part 01 of this booklet walks through that core idea in four short sections. You will learn the simple definition of an AI agent. You will see what makes it different from traditional software. And you will meet the loop that every agent runs, whether it controls a self-driving car, a workplace safety camera, or a chatbot that can click buttons on a webpage.

## 1. From Module 08 to Module 09

Module 08 gave you three pillars of modern computer vision. Vision Transformers, the architecture that processes an image as a sequence of patches with attention. Vision Language Models, which combine vision and language so a model can see an image and reason about it in natural language. And promptable vision tools like SAM, which respond to clicks and text instructions at runtime to segment objects on demand.

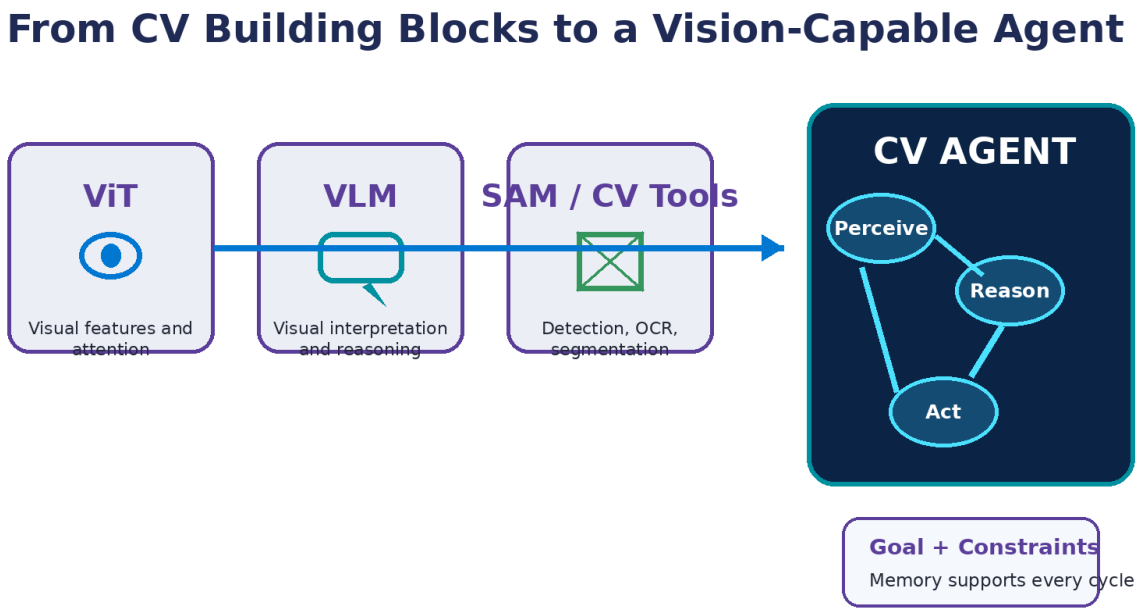
Each of those three pillars was a building block. On its own, a Vision Transformer is a classifier or a feature extractor. A Vision Language Model is a chatbot that can read images. SAM is a tool you call when you need a precise object outline. All three are useful. None of them, on their own, is an agent.

A vision-capable agent is a goal-directed system that coordinates these building blocks with tools, constraints, a repeating decision process, and—when the task requires it—memory. The developer still defines the goal, available tools, permissions, safeguards, success criteria, and stopping conditions. The agent then selects appropriate next steps within those boundaries.

### The Three Pillars You Will Reuse

As you read this booklet, you will see each of the three pillars from Module 08 show up again, in a new role. Vision Transformers serve as the visual feature extractor inside many agents. Vision Language Models serve as the reasoning engine that decides what to do based on what

was seen. Promptable tools like SAM serve as one of several visual tools the agent can call when it needs precise output. Same building blocks, new combinations. That is the entire shape of the upgrade.




The models are building blocks. The agent is the goal-directed system that selects tools, acts, and checks results.

*Figure 1. Module 08 models become components inside a goal-directed CV agent*

.

**KEY TAKEAWAY | Module 09 in one sentence**

Take the CV building blocks from Module 08, wire them into a loop with a goal and a memory, and you have a CV agent.

**INDUSTRY PRACTICE | Why this matters for your career**

Five years ago, a CV engineer spent most of their time training models. Today, more and more of the job is wiring pretrained models together into agents that solve real problems. Pretrained models are increasingly accessible. Employers value people who can evaluate them, connect them responsibly, and turn them into reliable systems that solve real problems.

## 2. What Is an AI Agent?

Here is the definition you should carry for the rest of the module, the lab, and the final project. An AI agent is a goal-directed system that observes its environment, chooses and carries out actions, checks the results, and maintains useful state or memory when needed.

That sentence is short on purpose. Five ideas matter: Goal, Perceive, Reason, Act, and Observe. Memory supports the process by carrying relevant information from one cycle to the next. An agent may adapt its next decision from feedback without permanently learning or retraining its model.

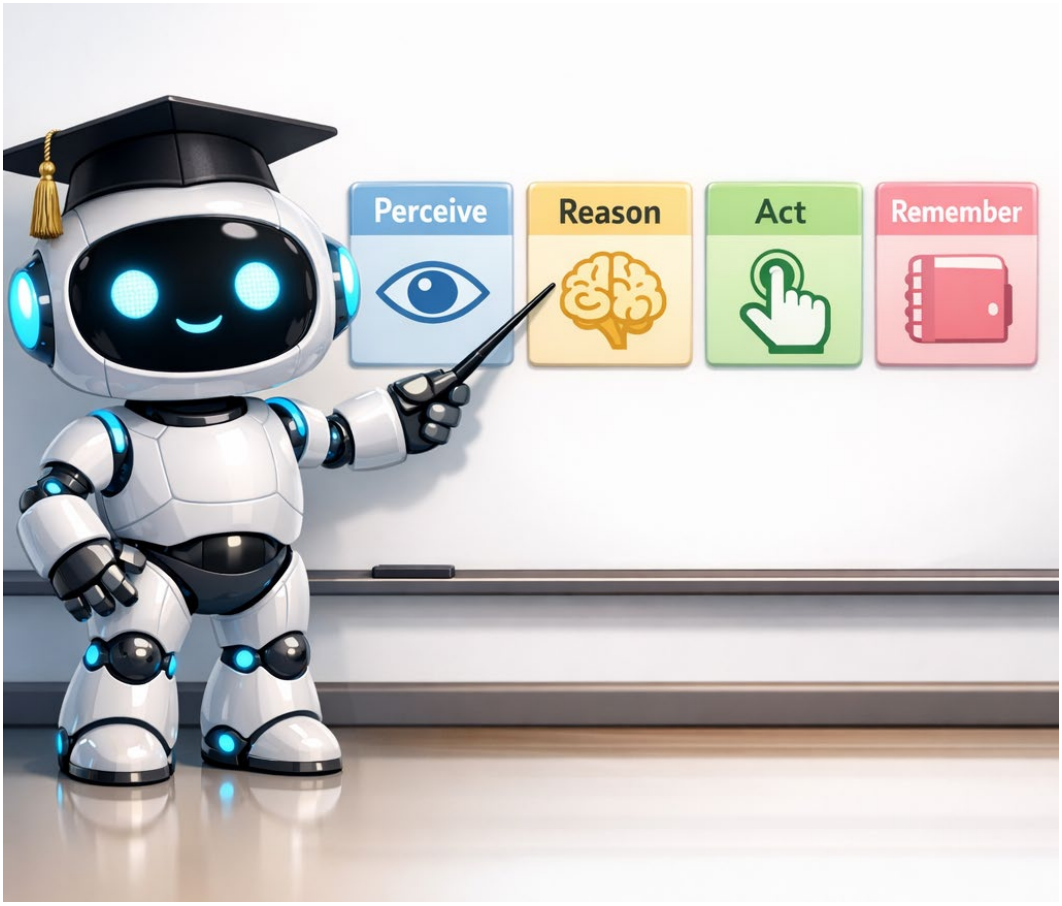



*Figure 2. Perception, reasoning, action, and observation form the cycle; memory supports the cycle rather than acting as a required final step.*

### Perceive

The agent gathers information about its environment. For a CV agent, perception means images, video frames, screenshots, scanned documents, anything the agent can see. The richer the perception, the more options the agent has when it reasons. A blind agent has very few options.

### Reason

The agent decides what action would best serve its goal. Reasoning is where the agent looks at what it perceived, considers what it knows from memory, and picks a next step. In modern CV agents, the reasoning step is almost always handled by a Vision Language Model that produces a sentence describing what to do next.

### Act

The agent does something. Sends an alert. Clicks a button. Saves a file. Steers a vehicle. Updates a database. Action helps distinguish a tool-using agent from a system that only produces responses. A chat interface may be connected to either kind of system.

### Remember

The system carries forward relevant information about what just happened. Memory can turn independent model calls into a continuous process by preserving recent observations, prior actions, and important history. Memory does not automatically make an agent smarter: inaccurate, irrelevant, or stale memory can also make decisions worse.

> **KEY TAKEAWAY | Four words, in this order**
>
> Perceive, Reason, Act, Observe—while memory supports the cycles. If you can name these four steps for any system you encounter, you can tell whether it is really an agent or just a model with a chat interface.

> ⚠ **MISCONCEPTION ALERT | A chatbot is not the same as an agent**
>
> A chat interface is not automatically an agent. A chat interface may only answer questions, or it may provide access to an agent that uses tools and performs actions. Always distinguish among the underlying model, the user interface, and the complete system. The same model can support a simple chat experience or a tool-using agent, depending on the surrounding software and permissions.

## 3. Agents Versus Traditional Software

To really see what an agent is, contrast it with the kind of software you have probably written or used your whole life. Traditional software is built from a long list of steps that a programmer writes in advance. The thermostat in your hallway runs traditional software. So does the cash register at a grocery store. So does the assembly line robot that bolts on a car door.

### How Traditional Software Works

A programmer sits down, thinks through every situation the program might face, and writes a step for each one. If the temperature drops below sixty eight degrees, turn on the heat. If the temperature rises above seventy four degrees, turn it off. The program follows that script forever. It is reliable, predictable, easy to test, and completely rigid. Drop it into a new situation, and it does not adapt. It just keeps running the script.

### How an Agent Works

With an agent, the developer usually does not script every possible decision in advance. Instead, the developer specifies the goal, available tools, constraints, permissions, checkpoints, and stopping conditions. For example, a building-management agent could consider time of day, weather, occupancy, and energy price while remaining inside clearly defined comfort and safety limits.

This flexibility is the whole point of building agents. It is also the source of all the new problems agents create. An agent that adapts can sometimes adapt in surprising ways. A traditional thermostat that fails fails predictably. An agent that fails can fail in ways no one anticipated. Both kinds of software have a place. The engineering skill is knowing which one to reach for.

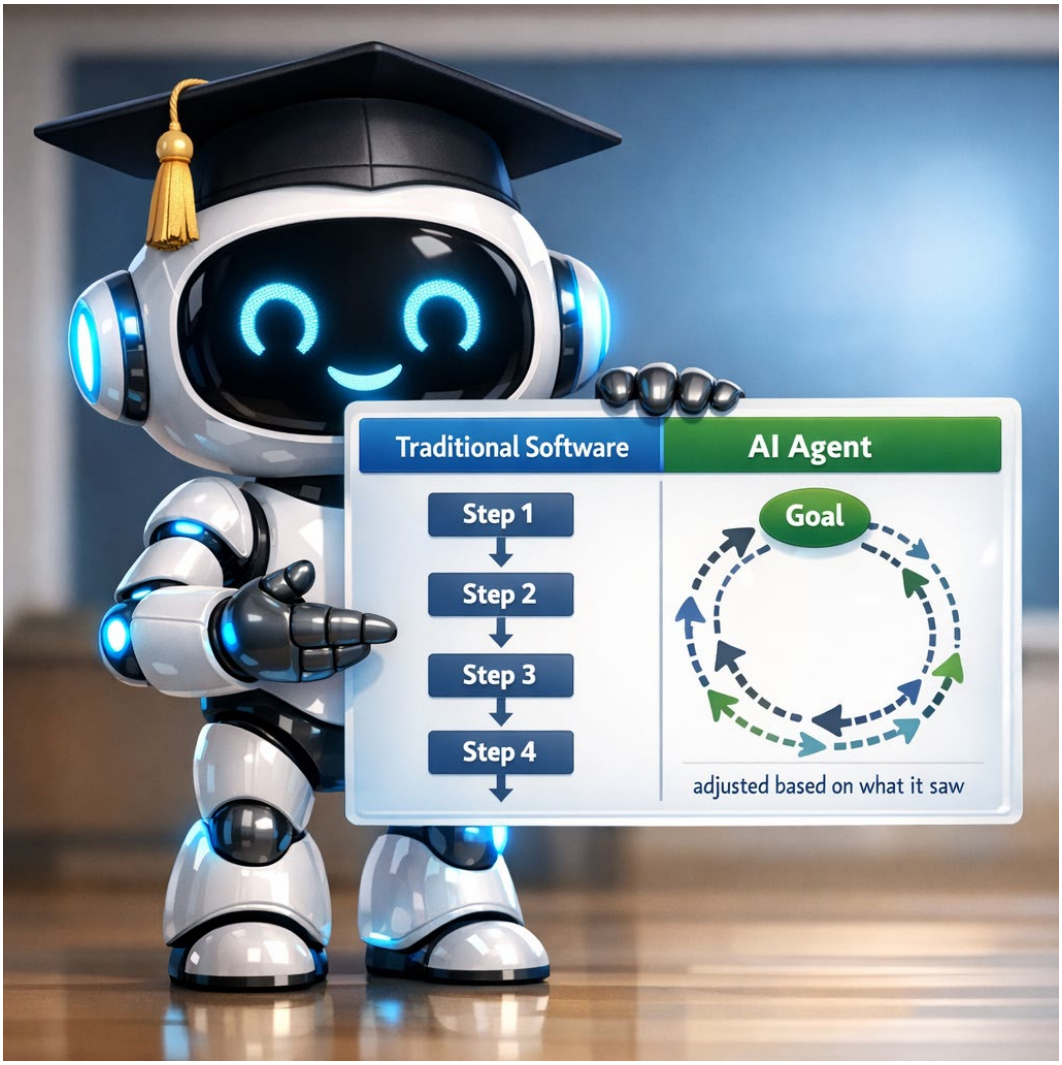



*Figure 3. Traditional software follows predefined logic; an agent can select among permitted actions while pursuing a goal*

**INDUSTRY PRACTICE | When traditional software is still the right answer**

On a real factory floor in 2026, the conveyor belt motor controllers still run traditional software. The robotic arm that welds car frames still runs traditional software. The check engine light still runs traditional software. These systems need to be deterministic, fast, and certified for safety. An agent is a poor choice when failure has to be predictable. Knowing where traditional software wins is part of being a useful engineer.

> 💭 **THINK ABOUT IT | A real workplace question**
>
> Pick one piece of software you use every day. Is it traditional software or an agent? How can you tell? If you cannot tell, what test would help you decide? This is the kind of question that comes up in interviews when employers want to see how you think about systems.

## 4. The Perception Reasoning Action Loop

Many agents can be understood as a repeating loop. The system perceives or receives information, interprets the situation, chooses an action, performs it, and observes the result. It then repeats until the goal is reached, a stop condition is met, or a human takes over.

This general idea appears under several names, including the perception-action loop and sense-plan-act. In this booklet, we use a simplified perception-reasoning-action view, while remembering that observation of the result closes the loop. Different agent architectures divide these steps differently.

### Why the Loop Matters

If you only had to do one perception step, you would not need an agent. A single image classification is a one-shot task. The world stays fixed, the model runs once, you have your answer. But the real world does not hold still. A pedestrian moves into the crosswalk. A worker reaches for a tool that was not there a moment ago. A document arrives in the inbox that needs to be filed. An agent matters because the world keeps changing, and the agent has to keep up.

### Where Vision Lives in the Loop

For a CV agent, computer vision lives in the perception step. Every cycle of the loop begins with the agent gathering visual information. The Vision Transformers, Vision Language Models, and promptable tools from Module 08 are all candidates for that step. The choice of which tool to use in perception depends on what the agent is trying to do. A safety monitor might call YOLO for fast detection. A document agent might call a VLM for rich understanding. A robot might call all three at once.

## Where Memory Fits In

Memory is a supporting capability rather than a required sequential step. It stores useful state between cycles and can inform perception, reasoning, tool selection, and duplicate prevention. Some short tasks need little or no persistent memory; long-running monitors usually need more.

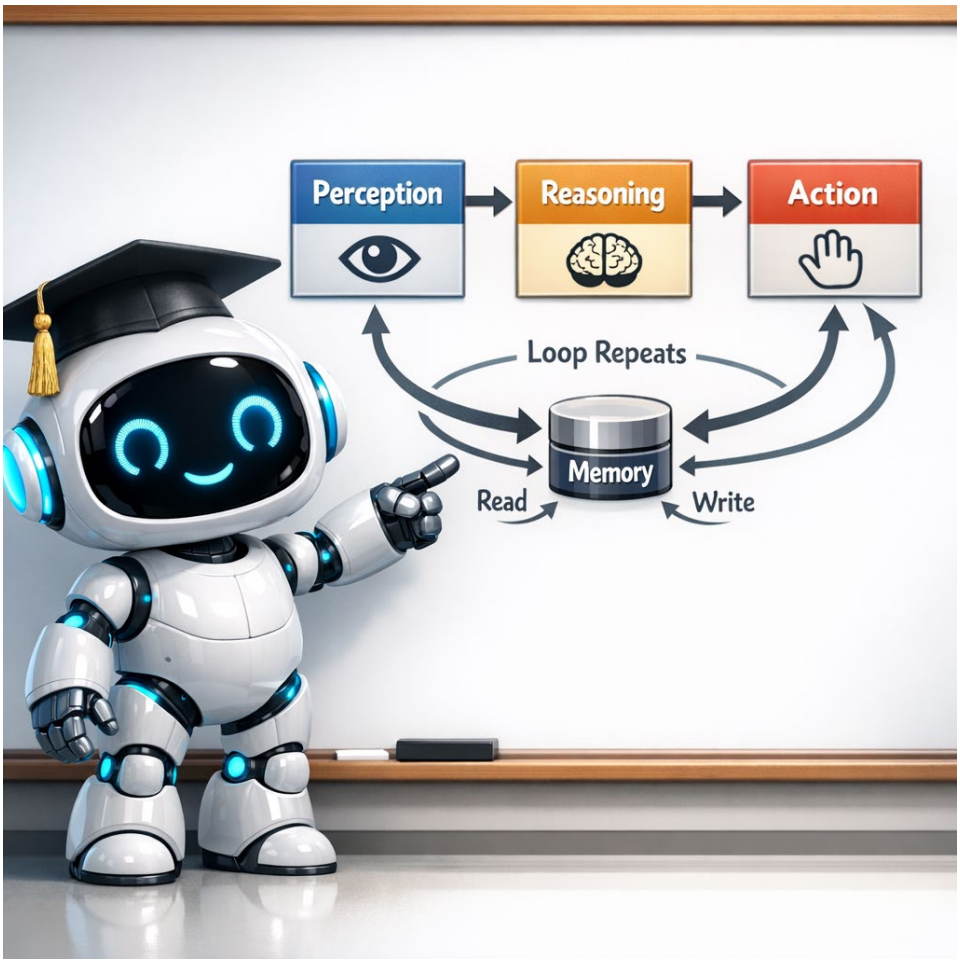



*Figure 4. A simplified agent loop. Memory stores relevant state between cycles.*

**KEY TAKEAWAY | The one diagram from this module**

Goal and constraints guide the process. Perceive, reason, act, and observe the result. Repeat as needed. Memory can preserve useful state between cycles. If you only carry one image away from Module 09, this is it.

### DEEPER LOOK | How fast does the loop run

The loop runs at a rate appropriate to the task and the available hardware. A real-time control system may update many times per second, while a workplace monitor may analyze selected frames and a document agent may run once per uploaded file. There is no universal rate, and language-model reasoning is often kept outside the fastest safety-critical control loop.

### KNOWLEDGE CHECK | Test yourself on Part 01

**1.** Define an AI agent in one sentence, using the words goal, see, decide, act, and learn.

**2.** Name the four core capabilities of an agent in the order they appear in the loop.

**3.** Give one example of traditional software and one example of an agent that you encounter in daily life.

**4.** Explain why memory matters for a CV agent watching a video feed.

# Part 02 | Computer Vision Inside an Agent

How your CV skills become an agent's eyes and brain

This is the heart of Module 09. Part 01 gave you just enough agent foundation to follow along. Part 02 zooms in on the specific question that matters to a CV student. Where does computer vision plug into an agent, and what does it unlock when it does?

The eight sections that follow walk through every place a CV skill becomes an agent's capability. You will see how vision serves as the perception layer, how Vision Language Models serve as the reasoning layer, how individual CV tools become tools the agent can call, and how memory turns a one-shot prediction into a continuous monitor. By the end of Part 02, you should be able to look at any AI product on the market and identify where its computer vision lives.

## 5. Computer Vision as the Eyes of an Agent

An agent that can process visual information can work with images, video, screenshots, charts, scanned documents, and physical scenes. Text and structured interfaces remain useful, but vision expands the range of situations the system can interpret.

### What Text Only Agents Cannot Do

Consider a few tasks: interpret a screenshot of an error, watch a security camera for a specific event, inspect a production line, or understand a document whose meaning depends on layout. Some web tasks can be solved through APIs, HTML, the DOM, or accessibility trees. Vision is especially valuable when meaning depends on graphical layout, images, charts, canvas elements, or controls that are difficult to interpret structurally.

Vision therefore removes an important limitation. It lets an agent combine text, structured data, and visual evidence instead of relying on only one input channel. The best design uses the simplest reliable source—API, document structure, OCR, detector, or VLM—for each part of the task.

## Where Your CV Skills Plug In

Every skill you have built in this course is part of the perception layer of a future CV agent. Knowing how to read pixels in Module 02 lets you understand what an agent's camera input looks like. The convolutional networks from Module 04 are still the backbone of fast object detectors. The Vision Transformers from Module 08 are the standard backbone for VLM-based reasoning. Image segmentation, object detection, image classification, all of these are tools an agent will call when it needs to see.


*Figure 5. Vision expands the range of environments and artifacts an agent can interpret.*

**KEY TAKEAWAY | Vision is not optional**

If you want to build agents that can do useful work in the real world, those agents need eyes. Vision is the bottleneck whose removal unlocks most of the value an agent can deliver.

**INDUSTRY PRACTICE | What this means for hiring**

In 2026 job postings, the phrase Computer Vision Engineer increasingly overlaps with the phrase AI Engineer or AI Agent Engineer. Recruiters are looking for people who can connect CV skills to broader systems. Knowing how to fine-tune a YOLO model is still valuable. Knowing how to embed that YOLO model into a larger agent is more valuable. The combination is what gets you hired.

## 6. VLMs in Interpretation and Reasoning

Once an agent can process visual information, the next question is how the system interprets what it observed and chooses an appropriate next step. A Vision Language Model can support both perception and reasoning, but it is only one possible component. Rules, planners, state machines, policy models, and specialized software may also participate.

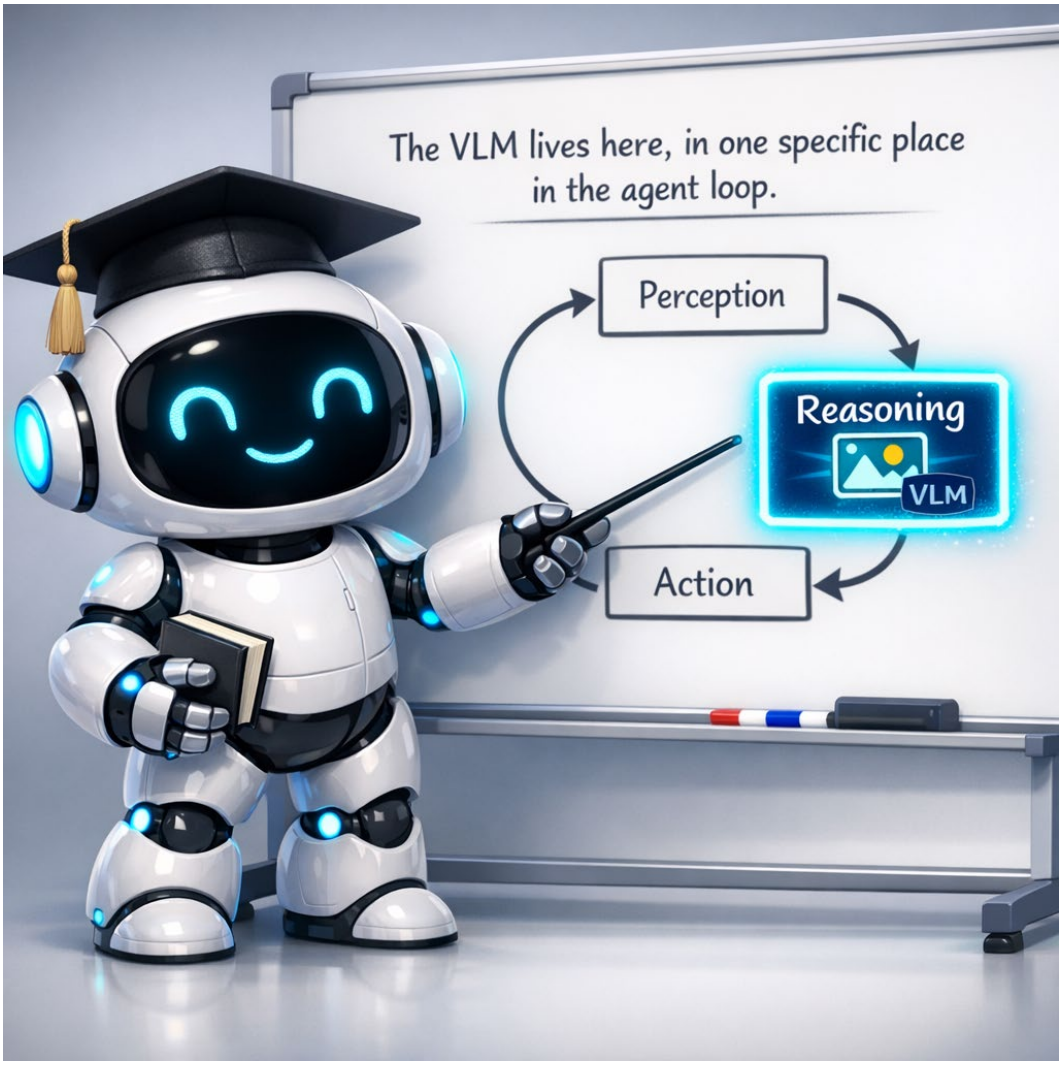



*Figure 6. A VLM can support interpretation and reasoning, but it is one component of the larger system.*

## The Leap from Labels to Sentences

A traditional computer vision model produces labels. A bounding box around an object with a class name attached. A segmentation mask. A confidence score. All useful, but flat. The output of a traditional CV model does not tell you what to do. It just tells you what is there.

A Vision Language Model can produce natural-language explanations, structured JSON, classifications, coordinates, tool calls, or recommended actions. For example, it may report that a worker without required PPE is near moving equipment and provide structured evidence for a downstream rule or human reviewer.


*Figure 7. Specialized detection provides grounded labels; a VLM can add contextual interpretation.*

The leap from isolated detections to contextual interpretation is one reason VLMs are useful in modern CV systems. A detector may locate a person, forklift, vest, and hardhat. A VLM or explicit rules can then assess relationships among those observations. For high-stakes tasks, the conclusion should be verified with task-specific checks and human review rather than accepted solely because a VLM produced a fluent explanation.

### Which Models Can Support Reasoning

Production systems may use a current frontier multimodal model from a commercial provider, an open-weight VLM hosted by the organization, or a hybrid of smaller specialized models and rules. The right choice depends on accuracy, privacy, latency, cost, deployment environment, and the consequences of an error. Exact model names change quickly, so students should consult current provider model cards and documentation.

Commercial frontier models are often easier to prototype with and capable across a wide range of edge cases. Open-weight models offer more control over hosting, privacy, and customization but require more engineering. Start with a measurable task and evaluation set; choose the model that meets the requirement rather than the model with the most publicity.

> **KEY TAKEAWAY | VLM is the brain**
>
> In a CV agent, a Vision Language Model can support visual interpretation and reasoning, but it is not the only possible reasoning component. It looks at what was perceived and produces a description that drives the next action. Choose carefully, but do not overthink it. Frontier models work well for most starting projects.

> ⚠ MISCONCEPTION ALERT | You do not need to fine-tune the VLM
>
> A common beginner instinct is to assume that every task requires fine-tuning. Often, a clear prompt, representative examples, a constrained output schema, and task-specific validation are enough for a first prototype. Fine-tuning becomes useful when evaluation shows persistent, repeatable failures that prompting, retrieval, or workflow changes do not solve.

## 7. What a Vision Capable Agent Unlocks

With perception and reasoning in place, an enormous range of tasks becomes possible. This section is a tour of what vision-capable agents are actually doing in 2026. Each item represents a growing application category that combines established CV skills with newer multimodal and agent-oriented techniques.


*Figure 8. Vision-capable systems can work with software interfaces, video, documents, and physical environments.*

### Reading the Web Visually

Web agents can interact through APIs, HTML, the DOM, accessibility trees, or visual screenshots. Vision is especially helpful when meaning depends on layout, images, charts, canvas content, or graphical controls. A robust system may switch among structured browsing, direct APIs, and visual interaction rather than relying on screenshots for every step.

### Watching Video Feeds

A vision-capable agent can watch a security camera continuously and alert when something specific happens. A worker enters a restricted zone. A safety violation occurs. A piece of equipment fails. A customer reaches a counter that is unstaffed. These are real applications running in factories, warehouses, hospitals, retail stores, and corporate campuses across the country. The work involves connecting the CV perception layer to a VLM reasoning layer to an alert action layer, with memory to avoid duplicate alerts.

### Processing Documents at Scale

Insurance claims, invoices, medical records, contracts, expense reports, government filings. Every document is a structured visual artifact that has to be read, classified, validated, and routed. A vision-capable agent does all of this in a few seconds per document. The agent reads the document with a VLM, extracts the structured fields, checks them against rules, and submits them to the next system in the pipeline. This is the largest single category of CV agent work by dollar volume in 2026.

### Navigating the Physical World

Robots, drones, and autonomous vehicles all use vision-capable agents. The agent perceives the environment through cameras, reasons about what it sees, and acts by moving in physical space. This includes industrial robots in factories, delivery robots on sidewalks, drones in agriculture, and the well-known case of self-driving cars. The same PRA loop runs in all of them, just at different speeds and stakes.

> **INDUSTRY PRACTICE | Where the jobs really are**
>
> All four categories can create employment opportunities. Document intelligence is a major enterprise application, autonomous systems are technically demanding, and video monitoring can offer accessible portfolio projects. Local opportunity varies by employer, regulation, and region, so use current job postings and conversations with industry partners to guide your plans.

> 💭 **THINK ABOUT IT | What would you build**
>
> Of the four categories described above, which one is closest to a job your future self might want? What would your first project in that category look like? You do not need a complete answer, just a starting point. The more concrete your target, the easier it is to direct your studying.

> 🔍 **DEEPER LOOK | Vision capable agents in accessibility**
>
> One category worth knowing about is accessibility. Visual-assistance systems can help blind and low-vision users understand scenes, documents, controls, and everyday objects. These applications also illustrate why clear descriptions, uncertainty handling, privacy, and user control matter.

## 8. The ReAct Pattern

ReAct, short for Reason plus Act, is one influential pattern for building tool-using agents. It was introduced in a 2022 research paper and organizes work by interleaving decisions, actions, and observations. Not every agent uses ReAct, but it is a useful foundation for understanding iterative tool use.

### The Three Step Cycle

A simplified ReAct cycle has three visible parts. First, the agent records a concise decision or reasoning summary about what to do next. Second, it takes an ACTION, often by calling a tool. Third, it receives an OBSERVATION containing the result. The observation informs the next decision. Production systems generally log concise decision summaries, tool calls, state changes, and errors rather than exposing private internal chain-of-thought reasoning.

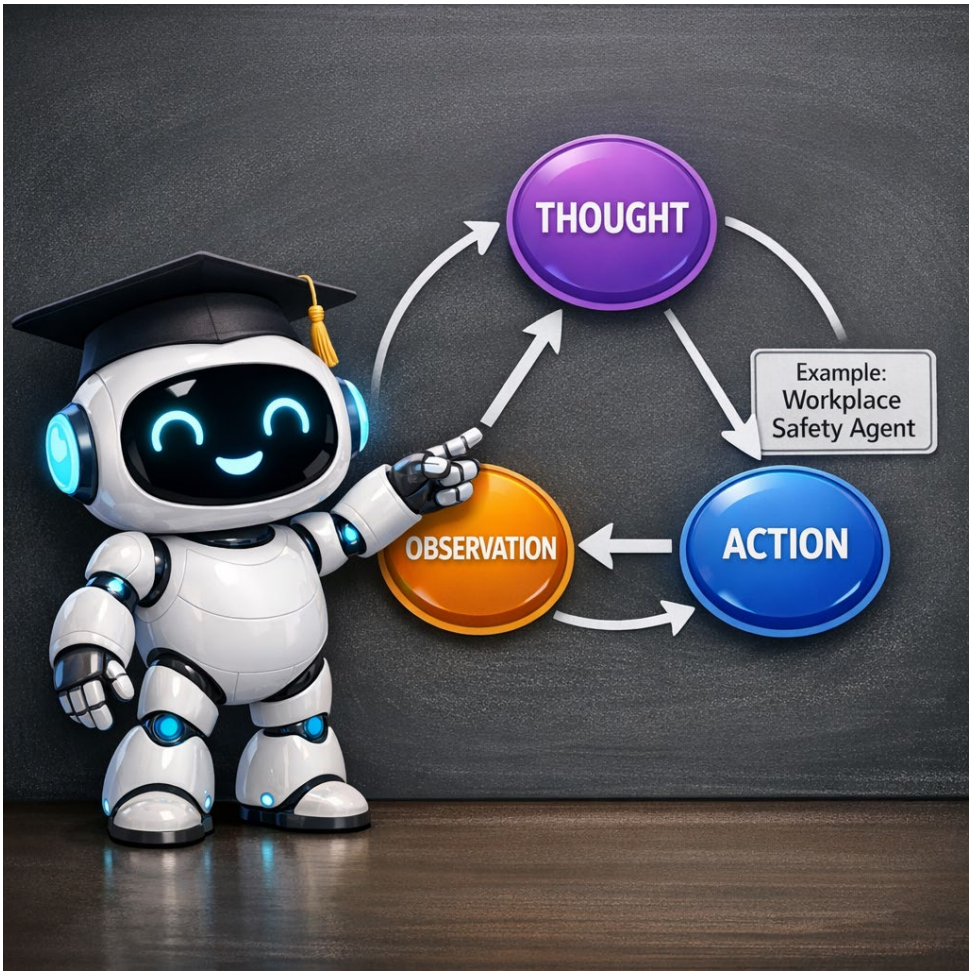



*Figure 9. ReAct alternates a decision or reasoning summary, an action, and an observation.*

That is the entire pattern. THOUGHT, ACTION, OBSERVATION, repeat. Every modern AI agent uses some version of this, whether the developer wrote the loop explicitly with code or used a framework like LangChain that builds the loop for them under the hood.

### Why the Pattern Is Useful

ReAct became influential partly because its tool-use traces are easier to inspect than an opaque one-shot response. When an agent fails, developers can examine the decision summary, selected tool, tool input, observation, state transition, and stop condition to locate the problem. ReAct is not the only debuggable architecture, but it provides a clear teaching model.

## ReAct in a Workplace Safety Agent

Here is what one simplified cycle of a workplace safety agent might look like. DECISION: Check this frame for people and required PPE. ACTION: Call a PPE-specific detector. OBSERVATION: One person detected; hardhat detected; safety vest not detected. DECISION: The evidence may indicate a PPE violation near machinery. ACTION: Add the frame to a human-review queue. OBSERVATION: Review item created and logged.

Notice that the system did more than detect objects. It combined evidence, applied a decision rule, selected an action, and confirmed that the action completed. In a real workplace, a human safety professional should review uncertain or consequential cases before disciplinary action is taken.

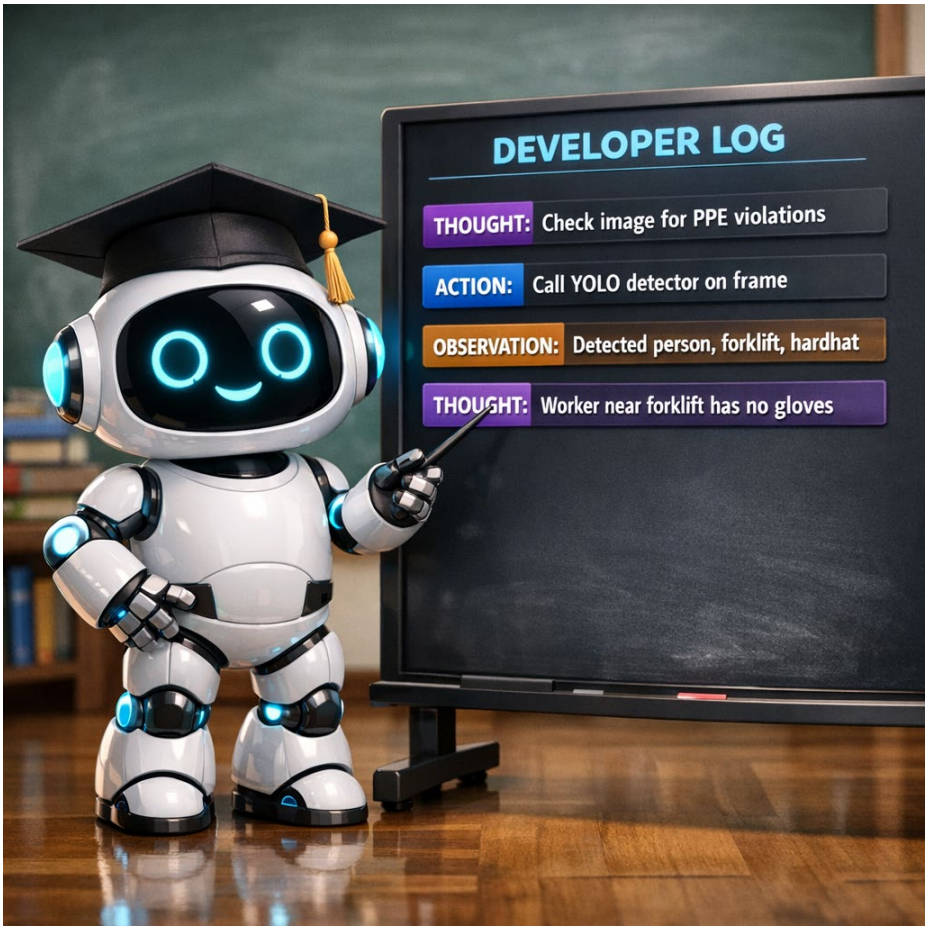



*Figure 10. An inspectable tool-use trace helps developers locate errors in decisions, tools, and observations.*

**KEY TAKEAWAY | ReAct is a pattern, not a product**

You will see ReAct and related decide-act-observe patterns in many tool-using agents, whether the developer talks about it explicitly or not. The frameworks come and go. The pattern stays. Once you can read a ReAct trace, you can debug almost any agent.

**DEEPER LOOK | Variations on the pattern**

Researchers and engineers have built many variations on the basic ReAct loop. Plan-and-Execute first builds a multi-step plan before acting. Tree of Thoughts branches into multiple parallel reasoning paths. Reflexion adds a step where the agent critiques its own past actions before deciding the next one. All of these are variations on the core idea of interleaving reasoning and action. If you continue to ITAI 2376, you will study these variations in depth. For ITAI 1378, knowing the basic ReAct pattern is enough.

## 9. Vision Tools Your Agent Will Call

In the ACTION step of the ReAct loop, the agent typically calls a tool. For a CV agent, that tool is usually a computer vision model that does one specific thing well. You have already met most of these tools in earlier modules. The skill that is new in Module 09 is knowing how to wire them into an agent and how to choose which tool to call when.

### Object Detection

Object detection finds objects in an image and draws a bounding box around each one. YOLO and its descendants are the standard tools. Object detection is fast, cheap, and reliable. A typical CV agent calls a detector first when it needs to know what is in a scene, then decides whether to look more carefully with a more expensive tool. Detection is the workhorse of CV agents.

### Segmentation

Segmentation draws a precise mask around an object rather than just a box. SAM and its successors are the standard tools, especially when the agent receives a prompt at runtime like a click or a text instruction. Use segmentation when bounding boxes are not precise enough, such as when measuring the actual area of an object, calculating distances from edges, or extracting clean cutouts for further processing.

## Text from Images

Optical Character Recognition, or OCR, extracts text from images. Modern OCR is mature and reliable. Every CV agent that touches documents calls OCR somewhere. Modern VLMs can also read text directly, often with better accuracy on complex layouts, so the line between OCR and VLM is blurring. Use OCR when you need fast, cheap text extraction with high confidence. Use a VLM when you also need to understand what the text means in context.

## Visual Reasoning

Visual reasoning is what you call when the question is not what is in the image but what it means. A VLM is the right tool here. The cost is higher than OCR or detection. The reasoning is incomparably richer. A well designed CV agent reserves the VLM for the questions only a VLM can answer.


*Figure 11. A CV agent may call different tools for detection, segmentation, OCR, and contextual interpretation.*

**INDUSTRY PRACTICE | How to choose tools in production**

Real CV agents in production rarely call only one tool. They build a cascade. Cheap fast tools first, expensive smart tools when needed. A safety monitor might call YOLO on every frame, OCR on the few frames where text is detected, and a VLM only on frames where YOLO flagged a possible incident. This kind of cost-aware design is what hiring managers look for when they ask about agent architecture. Always reach for the cheapest tool that can answer the question.

## 10. Memory in a CV Agent

Most individual model calls are stateless unless the surrounding application supplies prior context. A YOLO inference or segmentation call normally processes its current input and returns an output. The agent application—not the model alone—implements buffers, histories, databases, and retrieval mechanisms.

### Short Term Memory

Short term memory holds the information the agent has been collecting during the current task. What did I see in the last few frames? What action did I just take? Which tools have I already tried? Short term memory typically lives in the agent's running context, a buffer of recent observations and actions that the reasoning step uses to make its next decision.

For a CV agent watching a video feed, short-term memory might hold the last thirty frames worth of detection results. The agent uses that buffer to track objects across frames, decide whether something it just saw is new or persistent, and avoid acting on a momentary glitch in the camera feed.

## Long Term Memory

Long term memory holds history that persists across tasks and survives a restart. Has the agent seen this person before? Did it already send an alert about this incident today? What patterns of failure has it learned over the past month? Long term memory typically lives in a database or a vector store outside the agent's running context, ready to be queried when the reasoning step needs context that the current buffer does not contain.

A workplace safety agent with proper long-term memory will recognize that a forklift it sees today is the same forklift that triggered an alert last week. The agent can then either suppress a duplicate alert or escalate the alert because the same problem keeps happening. Without long-term memory, every day starts from scratch, every incident is treated as new, and the agent never learns.

## The Memory Hierarchy

The memory hierarchy of a CV agent often looks like this. The deepest, slowest layer is a database that holds days or months of history. Above that sits a vector store of embeddings that the agent can query for similar past situations. Above that is a working buffer of the last few minutes of perception. At the very top is the current observation. Each layer is faster and smaller than the one below it. The reasoning step pulls from each layer as needed.

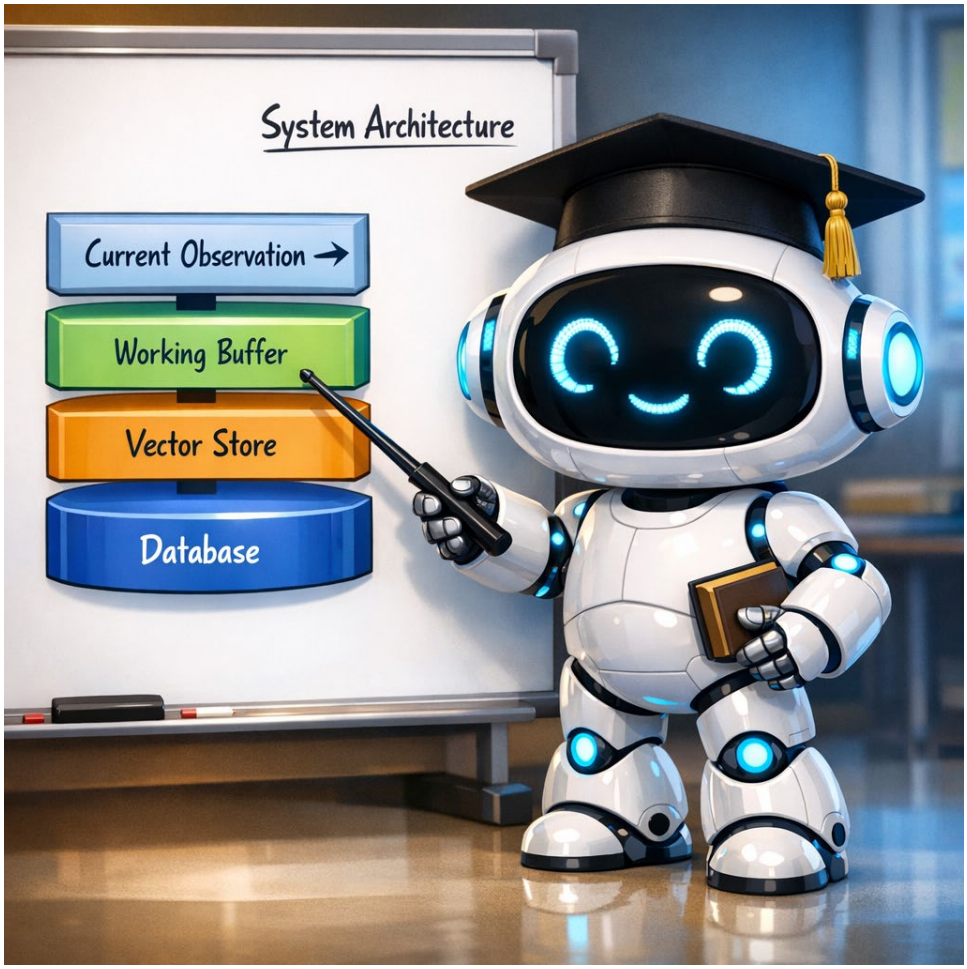



*Figure 12. Memory can range from the current observation to persistent databases and retrieval systems.*

**KEY TAKEAWAY | Memory is what turns a guess into a monitor**

Without retained state, a long-running CV system may treat each observation independently. With well-designed memory, it can track previous actions, reduce duplicate alerts, and compare current events with relevant history. Memory must be evaluated because stale or incorrect records can also cause errors.

**INDUSTRY PRACTICE | Where memory bugs live**

In production, memory and state-management bugs are common sources of agent misbehavior. The system may forget that it already sent an alert, confuse two tracked objects, retrieve irrelevant history, or preserve an incorrect conclusion. When debugging, inspect memory alongside perception, prompts, tools, and action logs.

**DEEPER LOOK | How production agents actually store memory**

In a production CV agent, short-term memory typically lives in a simple Python data structure or a Redis cache that the agent reads and writes during each cycle. Long term memory typically lives in a combination of a relational database for structured records (alerts, timestamps, identities) and a vector database like Pinecone, Weaviate, or pgvector for similarity search across past visual scenes. The reasoning step queries both layers as needed. You do not need to memorize these tool names for ITAI 1378, but if you ever interview for an agent role, knowing that the memory layer is a separate engineering concern from the model itself will set you apart.

## 11. Models and Tools for Visual Computer Use

A growing category of models and tools is designed for visual computer use. These systems parse screenshots, identify interactive elements, predict cursor or keyboard actions, and complete multi-step workflows in software. Some are specialized screen parsers, some are GUI-agent models, and some are complete products built from several models and tools.


*Figure 13. Screen parsers, GUI models, and full computer-use products occupy different layers of the visual-agent stack.*

## Microsoft OmniParser

OmniParser is a screen-parsing component for GUI agents rather than a complete autonomous agent. It converts a user-interface screenshot into structured, grounded elements such as buttons, icons, text fields, and bounding boxes. An outer model or agent can use that representation to decide where to click or type.

## ByteDance UI-TARS

UI-TARS is a GUI-agent model and software stack designed to interpret screenshots and produce computer actions such as moving the cursor, clicking, typing, or scrolling. Its surrounding desktop tools provide the execution environment, while safety controls and evaluation remain the developer's responsibility.

### Anthropic Computer Use

Anthropic Computer Use provides tools through which a Claude-based system can inspect screenshots and request mouse and keyboard actions in a controlled environment. The surrounding application executes the actions, enforces permissions, and should require human confirmation for sensitive operations.

#### OpenAI ChatGPT Agent

OpenAI integrated capabilities originally developed for Operator into ChatGPT agent. ChatGPT agent can combine visual and text-based browsing, direct tools, code execution, and connected data sources to complete multi-step tasks while keeping the user involved in consequential actions.

**KEY TAKEAWAY | A new category, not a passing trend**

Visual computer-use systems are a distinct application category, but they are not all the same kind of model. A screen parser, GUI-control model, general multimodal model, browser tool, and full agent product play different roles. The product names will change; the architectural distinctions are more durable.

**INDUSTRY PRACTICE | What employers expect you to know**

Employers may expect candidates for applied AI roles to recognize the category, explain its components, and discuss safety, evaluation, and human oversight. A small sandboxed demonstration can be valuable, provided that it does not automate sensitive actions or expose private data.

**⚠ MISCONCEPTION ALERT | Agent VLMs do not replace general VLMs**

A common student assumption is that specialized GUI models will replace general multimodal models. More often, they complement one another. A real system may use a screen parser for grounding, a multimodal model for interpretation, a workflow engine for state, and a controlled tool layer for execution.

> 💭 **THINK ABOUT IT | A real workplace question**
>
> Pick a tedious computer task you do at least once a week. Could one of the agent-oriented VLMs in this section automate that task? What part of the task would you find easy to hand over, and what part would still need human judgment? This kind of thinking is exactly what hiring managers want to hear about in interviews.

## 12. Frameworks: Names to Know

In any conversation about AI agents, the same handful of framework names will come up. These are the toolkits that handle the plumbing of building an agent, so the developer can focus on the logic. For ITAI 1378, you do not need to master any of these frameworks. You need to recognize the names and understand at a high level what each one does. If you continue to ITAI 2376, you will get hands on practice with several of them across three full modules.

### LangChain

LangChain provides higher-level components and prebuilt patterns for connecting models, prompts, retrieval, tools, and agent loops. It can speed up prototyping, but teams may also build directly with provider SDKs or other frameworks.

### LangGraph

LangGraph is a lower-level orchestration framework and runtime for stateful workflows that need branches, loops, persistence, retries, and human-in-the-loop steps. It can be used with LangChain components, but it is conceptually distinct from a simple linear chain.

## AutoGen and CrewAI

AutoGen and CrewAI are frameworks that can support coordination among multiple agents or roles. They can help structure supervisor-worker patterns, delegated tasks, and conversations among specialist components. Multi-agent designs add complexity and should be used only when separate roles provide measurable value.

## MCP, The Model Context Protocol

MCP is not an agent framework. It is an open standard for connecting AI applications with tools, resources, prompts, and external systems through a consistent interface. It has broad ecosystem support, but using MCP does not automatically make an application agentic, secure, or reliable.

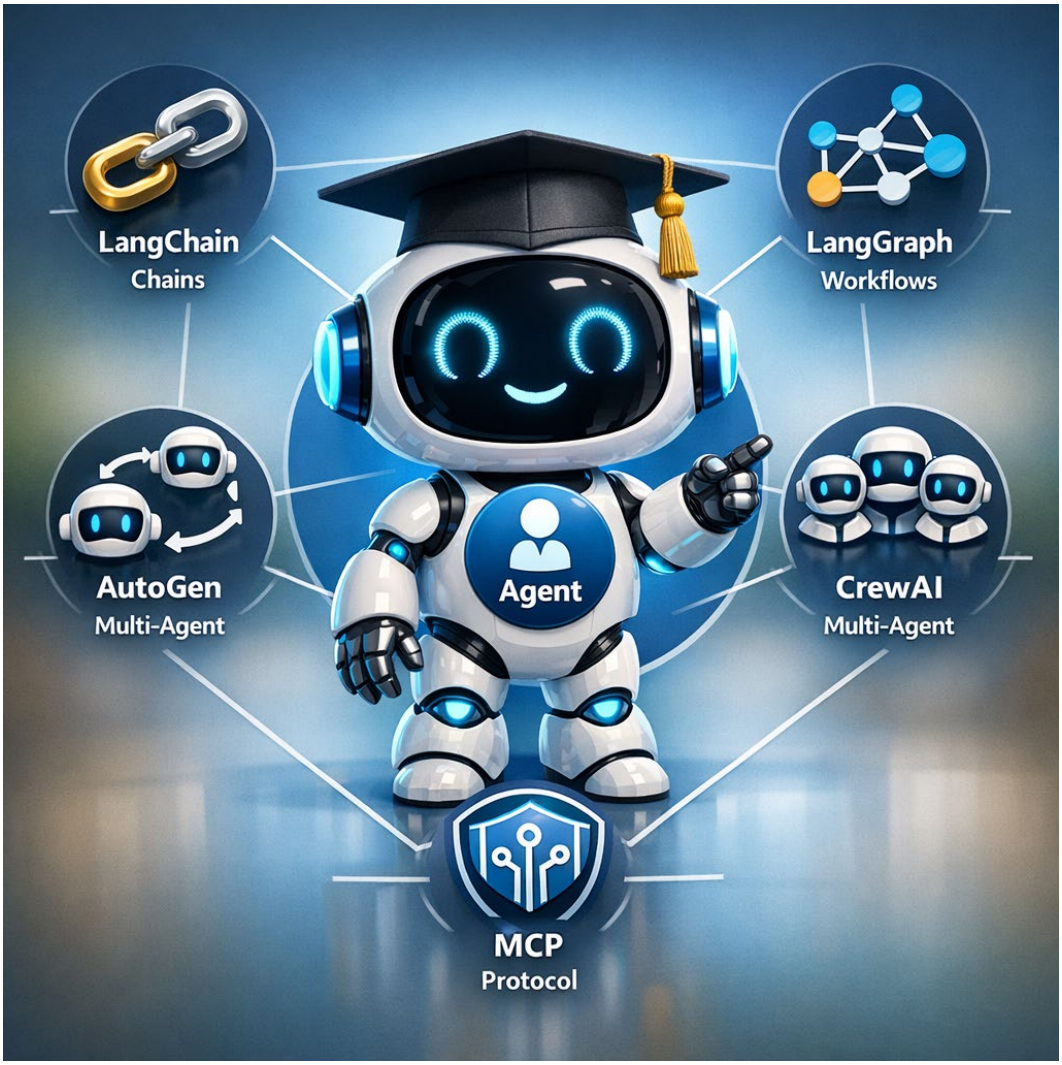



*Figure 14. Frameworks and protocols organize workflows and connections, but they do not replace task design and evaluation.*

**KEY TAKEAWAY | Names, not depth**

For Module 09, the goal is recognition. You should be able to say what each framework does in one sentence. Deep coverage is not the job of this course. If you want depth, take ITAI 2376.

**⚠ MISCONCEPTION ALERT | Frameworks are not magic**

A common misconception is that choosing a framework is the most important agent-design decision. It is not. The harder work is defining the task, selecting and evaluating perception tools, controlling permissions, designing state and recovery, validating outputs, and handling failures. A framework organizes implementation; it does not solve the engineering problem for you.

**KNOWLEDGE CHECK | Test yourself on Part 02**

1. Why does adding vision expand the tasks an agent can perform?
2. Name one role a Vision Language Model can play inside a CV agent.
3. List three vision tools an agent might call, with one use case for each.
4. Explain the difference between short-term and long-term memory in a CV agent.
5. Name two of the 2026 agent-oriented VLMs and what each one does.
6. Why are frameworks like LangChain useful, but not the most important design decision when building an agent?

## 13. Evaluating and Safeguarding a CV Agent

A convincing demonstration is not the same as a reliable system. A CV agent must be evaluated as a complete pipeline: perception, interpretation, memory, action tools, and human oversight. Each component can fail in a different way, and a fluent explanation does not prove that the underlying observation was correct.

### What to Measure

Useful measures include detection precision and recall, false-positive and missed-incident rates, duplicate-alert rate, end-to-end latency, API or compute cost, tool-call success, and the percentage of cases that require human review. Build a small evaluation set before changing prompts or models so that improvements can be measured rather than guessed.

### Privacy, Security, and Human Oversight

Visual systems may capture faces, badges, medical information, computer screens, or private work practices. Define what data is collected, who can access it, how long it is retained, and how people can challenge an incorrect result. Treat screenshots, documents, and webpages as potentially untrusted input because they can contain instructions designed to manipulate an agent. Restrict tool permissions, require confirmation for consequential actions, and keep an audit log.

### Workplace Fairness

Before deploying a workplace-monitoring system, test performance across lighting conditions, camera angles, clothing, PPE styles, body types, and relevant worker populations. The agent should support safety professionals, not make unreviewed disciplinary decisions. Workers should be informed about the system's purpose, limitations, escalation process, and data-retention policy.

KEY TAKEAWAY | Evaluate the whole system

A CV agent is only as trustworthy as its weakest component. Measure errors, limit permissions, protect visual data, and keep a human in the loop when decisions affect safety, employment, money, or legal rights.

# Part 03 | CV Agents in the Real World

Where CV agents actually run and earn their keep

Part 02 covered how a CV agent works. Part 03 zooms out and shows where these agents actually live in industry. The four sections in this part cover the three biggest application categories you are likely to encounter as a junior CV engineer, plus a section on when not to use an agent at all.

None of the examples in Part 03 require you to invent new models or train anything from scratch. Every one of them is built by combining the tools you already know about in smart ways. That is what a working CV agent engineer does, ninety percent of the time.

## 14. Industry Example: Autonomous Driving

Autonomous driving is a useful example of a perception-planning-action system operating under strict latency and safety constraints. Its architecture differs substantially from a language-model-based ReAct agent, but the general idea of observing the environment, updating state, planning, acting, and checking results is instructive.


*Figure 15. Autonomous driving combines perception, tracking, planning, deterministic control, and safety engineering.*

## Tesla as a Case Study

Tesla provides a familiar camera-based case study, while other autonomous-driving systems combine cameras with radar or LiDAR. Public descriptions emphasize neural-network perception, planning, control, and specialized onboard inference hardware. Exact sensor counts and internal architectures change across vehicle generations and software releases, so this booklet focuses on the durable system-level concepts rather than a fixed specification.

In a simplified view, the perception system identifies lanes, signs, vehicles, pedestrians, and free space. Prediction and planning estimate how the scene may change and choose a safe trajectory. Control software converts that plan into steering and braking commands. State estimation and tracking connect observations across time. Safety-critical control also includes deterministic checks and redundancy rather than relying on a VLM in the critical path.

### How Often the Loop Runs

Autonomous systems update perception and control many times per second, with different subsystems operating at different rates. The exact timing depends on hardware, sensors, models, and the control function. This tight latency budget explains why production systems use optimized models, specialized hardware, and carefully engineered safety layers.

### Why It Matters for You

Autonomous-driving teams often require specialized engineering experience, but the broader transportation sector also hires for fleet analytics, dashcam analysis, traffic monitoring, vehicle inspection, logistics, and driver-assistance systems. These adjacent areas can provide practical entry points for a strong CV portfolio.

**DEEPER LOOK | Beyond Tesla**

Other major players in autonomous driving include Waymo, which uses LiDAR alongside cameras, Cruise, Mobileye, and increasingly Chinese manufacturers like BYD and XPeng. Each company makes different architectural choices about sensors, models, and the safety layer. The PRA loop is the same. The implementation differs in interesting ways. If you are curious, the SAE levels of automation (zero through five) provide a useful framework for comparing how much of the loop each system claims to automate.

**INDUSTRY PRACTICE | Adjacent jobs you can actually get**

Even if you do not work on the autonomous vehicle itself, the broader transportation industry hires CV engineers to work on adjacent problems: fleet management, dashcam analysis, traffic monitoring, vehicle inspection at ports, license plate recognition for tolling. These are growing markets where a community college graduate with a strong portfolio can compete effectively. Look for these jobs as an entry point if you are interested in the space.

## 15. Industry Example: Workplace Safety

Workplace safety monitoring is an important applied market for computer vision. Factory floors, warehouses, construction sites, hospitals, and processing plants may use vision systems to flag possible hazards, support audits, and help safety professionals prioritize review.

### What the Systems Actually Do

A workplace safety agent typically watches one or more camera feeds continuously. The agent detects people, equipment, and required personal protective equipment such as hardhats, vests, gloves, safety glasses, and steel toed boots. When the agent observes a violation, missing PPE, an unauthorized person in a restricted zone, an unsafe distance between a worker and moving equipment, it sends an alert to a supervisor dashboard or to a wearable device worn by the safety officer.


*Figure 16. Workplace monitoring should flag possible events for qualified human review.*

The same agents are also used for non safety tasks like productivity counting (how many units passed the inspection station this shift), ergonomic analysis (which workers are using poor lifting posture), and equipment monitoring (which machine has been idle for too long). The same camera and the same underlying agent often does multiple jobs.

### Why This Market is Stable

Workplace safety is shaped by regulation, internal policy, insurance requirements, and the duty to protect workers. Vision systems can support documentation and review, but organizations must also address privacy, worker communication, bias, retention, cybersecurity, and the risk of false alerts.

### How to Get a Foot in the Door

Workplace safety can be a practical portfolio direction because the components are well documented and a small demonstration can be built with sample images, a PPE-specific detector, and a review dashboard. Students should present the project as decision support, describe limitations clearly, and avoid testing on people without permission.

**INDUSTRY PRACTICE | Houston specific opportunities**

Houston has substantial oil and gas, petrochemical, port logistics, healthcare, manufacturing, and aerospace activity. These sectors can create demand for inspection, monitoring, document intelligence, and safety-support systems. Current opportunities should be verified through employer partnerships and job postings rather than assumed from industry size alone.

**⚠ MISCONCEPTION ALERT | Safety agents augment, they do not replace**

A common misconception, both inside the industry and outside it, is that AI safety monitoring will replace safety officers. It will not, at least not for the foreseeable future. The agent flags possible violations. A human safety officer reviews the flags, decides which ones are real, and takes the actions that affect people's jobs. The agent makes the safety officer faster, more consistent, and better documented, not obsolete. When you talk

about these systems with potential employers, emphasize the augmentation framing. It is both more accurate and more politically realistic.

## 16. Industry Example: Document Processing and Retail

Document processing and retail are two additional categories of applied vision work. Document intelligence is common in enterprises with large volumes of forms and records. Retail applications range from visual search to shelf monitoring and inventory support.

### Document Processing

Every business runs on documents. Invoices, contracts, claims, medical records, expense reports, shipping manifests, regulatory filings. A document processing CV agent reads incoming documents, extracts the structured fields, validates them, and routes them to the next system. The agent uses a VLM for the actual reading, optionally OCR for high volume text extraction, and a database for memory of past documents and known patterns.

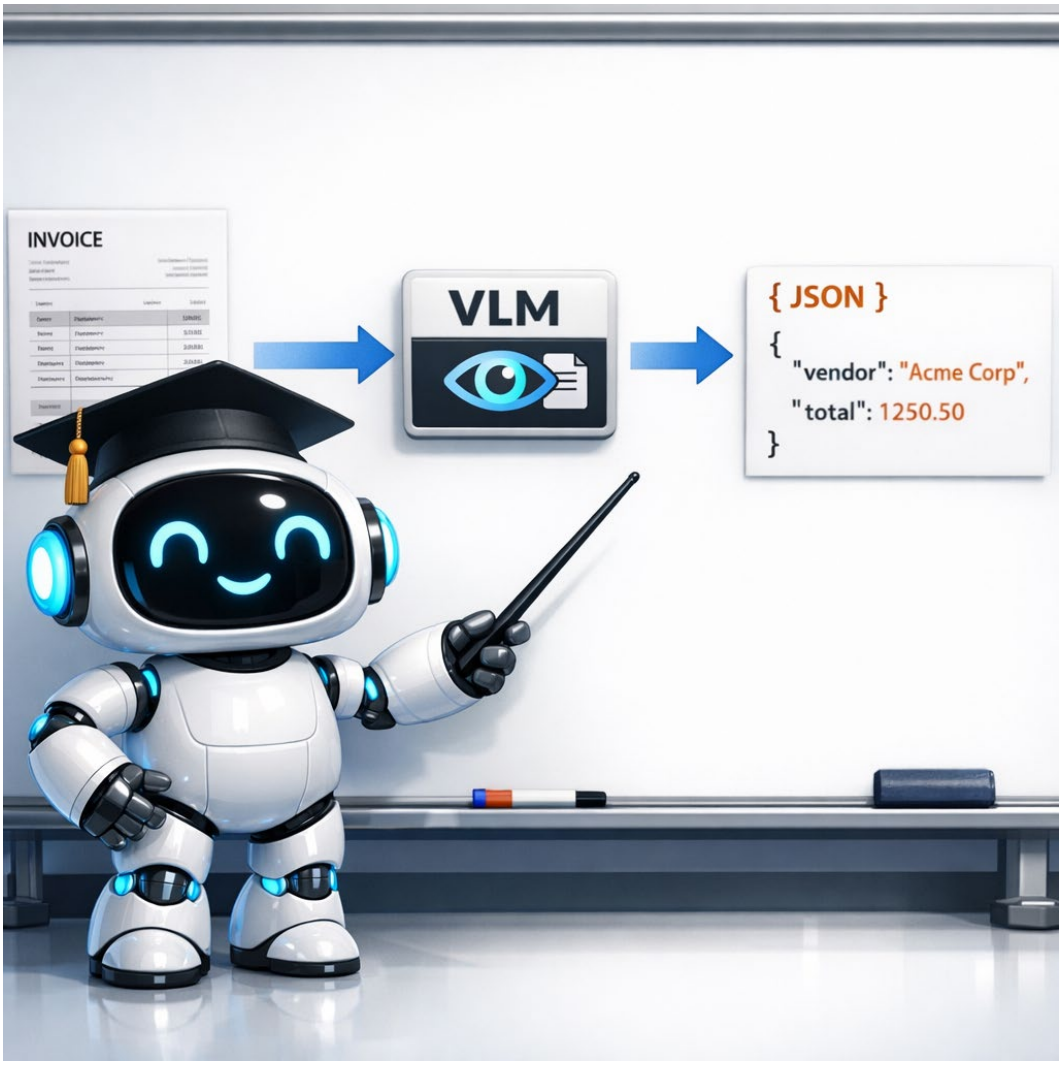



*Figure 17. A document-intelligence workflow converts visual input into validated structured data.*

Banks, insurers, healthcare networks, law firms, government agencies, and logistics companies use document-intelligence systems. Job titles vary, so students should search for terms such as document intelligence, intelligent document processing, multimodal AI, AI solutions engineering, and automation.

## Retail

Retail is more varied. Visual search lets a customer take a photo of a product and find similar items in the catalog. Shelf monitoring uses cameras in stores to detect stockouts, misplaced products, and pricing errors. Self checkout systems use vision to verify that the item being scanned matches the barcode being scanned. Loss prevention systems flag suspicious patterns of behavior on camera. Inventory robots autonomously drive aisles at night counting stock. Each of these is a CV agent.


*Figure 18. Retail vision systems may support shelf availability, placement, and pricing checks.*

Large retailers and specialist technology vendors maintain teams working on visual search, shelf analytics, inventory, checkout verification, and loss prevention. The work combines CV engineering with operational knowledge, data governance, and careful evaluation of false positives.

**INDUSTRY PRACTICE | Which category should you target**

Use these examples as directions rather than rankings. Choose a domain that matches your interests, local opportunities, and current skills. Then build a small, well-evaluated project that demonstrates the complete workflow and explains where human judgment remains necessary.

## 17. When NOT to Reach for an Agent

Agents are powerful. They are also not always the right tool. Knowing when to use an agent and when to use something simpler is one of the most valuable engineering skills you can build. This section walks through the cases where you should pause before reaching for an agent.

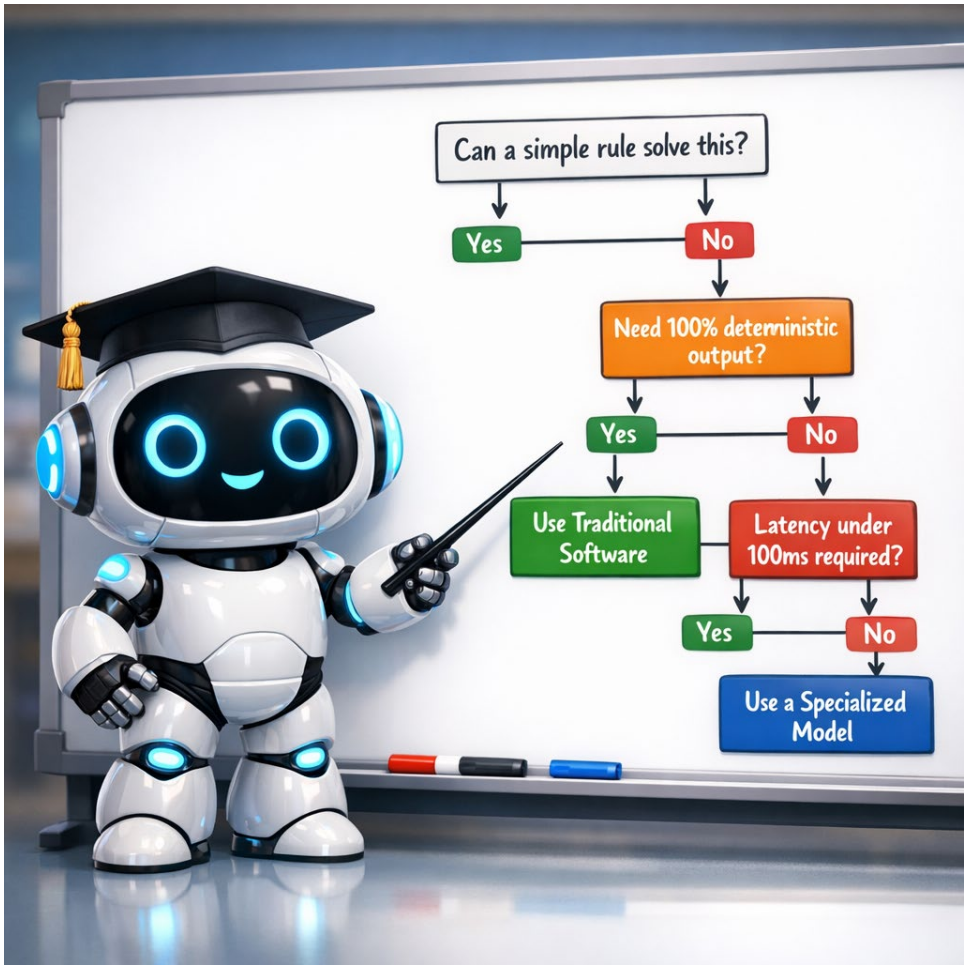



*Figure 19. Prefer the simplest reliable architecture that satisfies accuracy, latency, cost, and risk requirements.*

### When Simple Rules Already Work

If the problem can be solved with a clear if-then rule, use the rule. A traditional thermostat does not need an agent. A barcode scanner does not need an agent. A check engine light does not need an agent. Rules are faster, cheaper, more reliable, and easier to debug. Reach for an agent only when rules are clearly not enough.

### When You Need Determinism

Language-model-backed agents are probabilistic and may produce different outputs across runs. Even with low-temperature settings and structured schemas, complete reproducibility is difficult. When a requirement demands deterministic, certifiable behavior, use traditional software or a tightly constrained model and keep any generative component outside the critical decision path.

### When Latency Is Tight

VLM calls add latency and may take from fractions of a second to several seconds depending on the model, input, network, and workload. When a control loop has a very small time budget, use optimized perception and deterministic control in the critical path, with slower multimodal reasoning operating asynchronously if appropriate.

### When the Budget Is Tight

Commercial VLMs usually charge by input and output usage, while self-hosted models require hardware, energy, maintenance, and engineering. At scale, evaluate the total cost per successful task. A cascade can use fast, inexpensive tools first and reserve a larger VLM for uncertain or high-value cases.

### When the Stakes Are High and the Trust Is Low

If a wrong decision by the agent could cost a life, a lawsuit, or a customer relationship, the bar for trust is high. New agent technology has not yet earned that trust in many domains. For now, agents in those domains usually run in advisory mode, suggesting actions for a human to approve, rather than acting on their own. Knowing where this line currently sits in your industry is part of the job.

> **KEY TAKEAWAY | Smart engineers know when not to use an agent**
>
> The cheapest, fastest, most reliable system that solves your problem is the right system. If a rule does the job, use the rule. If a simple model does the job, use the simple model. Reach for an agent only when nothing simpler will work, and be ready to defend the choice in front of your manager.

> **THINK ABOUT IT | A question for your future job**
>
> Imagine your first day at a future employer. The team is debating whether to build a new feature as an AI agent or as traditional software. What three questions would you want to ask before you took a side?

### ☑ KNOWLEDGE CHECK | Test yourself on Part 03

**1.** Name three industries that hire CV agent engineers in 2026, with one example application for each.

**2.** Why is workplace safety often the most realistic first job for a community college graduate?

**3.** Give two situations where an agent is the wrong choice and traditional software is better.

**4.** Explain why frontier VLMs can be too expensive at scale, and what you can do about it.

# Wrap Up | Lab and What Comes Next

From booklet pages to keyboard practice

## 18. Connection to Your Final Project

Module 09 is the last module before final projects begin, and it gives you the structure for any CV project you might build. Whether you target a simple project or an ambitious one, the PRA loop, the ReAct pattern, and the tool selection ideas from this booklet apply. This section sketches three tiers of project ambition you can choose between.

### Tier 1, Simple CV System

A Tier 1 project uses a pretrained CV model, simple rule-based logic on top of the model output, and a basic alert or output mechanism. No VLM is involved. No framework is required. The PRA loop runs in the simplest possible form: perceive with the CV model, reason with hard-coded rules, act by printing or saving the result. Tier 1 is a perfectly respectable final project. The bar is execution quality, not novelty.

### Tier 2, VLM Enhanced Agent

A Tier 2 project adds a VLM or multimodal model to interpretation and decision support. A specialized CV model can still perform efficient perception. The VLM reviews selected evidence and returns a constrained result, while explicit rules and a simple memory layer manage triggers and duplicate prevention. A decide-act-observe trace provides a clear structure for debugging.

### Tier 3, Multi Agent System

A Tier 3 project coordinates several specialist components or agents only when separate roles add clear value. For example, one component may detect PPE, another may summarize incidents, and a supervisor workflow may combine their results. Multi-agent systems add communication, state, and debugging complexity, so discuss scope with the instructor before committing.

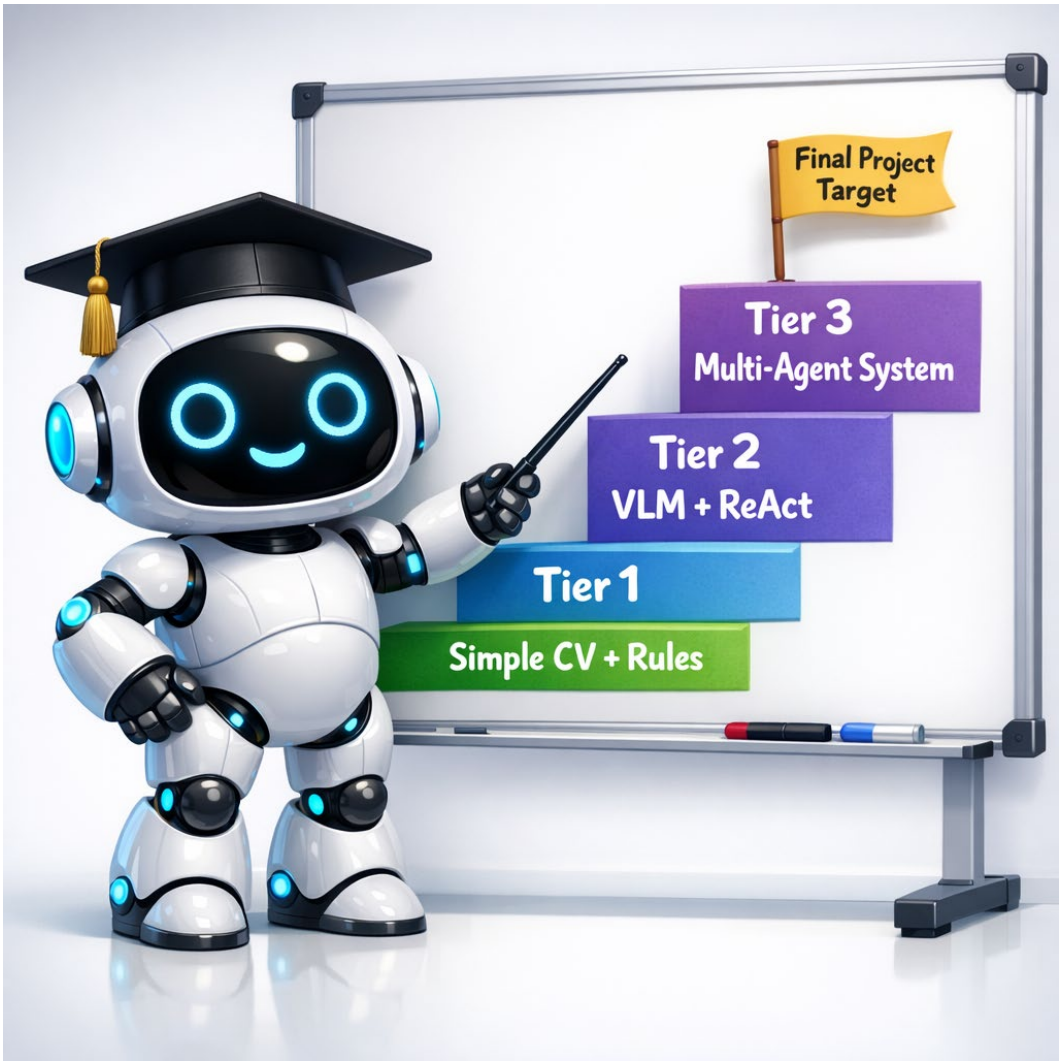



*Figure 20. Project scope should match available time, skills, and evaluation capacity*

**KEY TAKEAWAY | Pick the right tier for your level**

There is no extra credit for choosing a harder tier and delivering it poorly. A clean, well documented Tier 1 project is more impressive than a half finished Tier 3 project. Pick the level that matches your time, your skills, and your willingness to debug.

**INDUSTRY PRACTICE | What employers want to see**

When a hiring manager looks at a portfolio piece, they look for three things. First, can you explain what the project does in two sentences? Second, can you show a video of it working? Third, can you talk about what you learned from the failures? Notice that none of these care which tier you targeted. Quality of thought beats ambition of scope, every time.

## 19. Lab Preview

The Module 09 lab is your chance to put hands on the ideas from this booklet. Detailed instructions and starter code live in the lab notebook. This section gives you the high level shape of the work.

### What You Will Build

You will build a simple workplace safety monitoring agent using a small set of sample factory-floor images. The agent will use a pretrained PPE-specific YOLO model—or a model already fine-tuned for classes such as people, hardhats, and safety vests—to produce detections. A VLM will review selected evidence using a constrained prompt and structured output. The system will add possible violations to a human-review queue, and a simple memory layer will prevent duplicate review items.

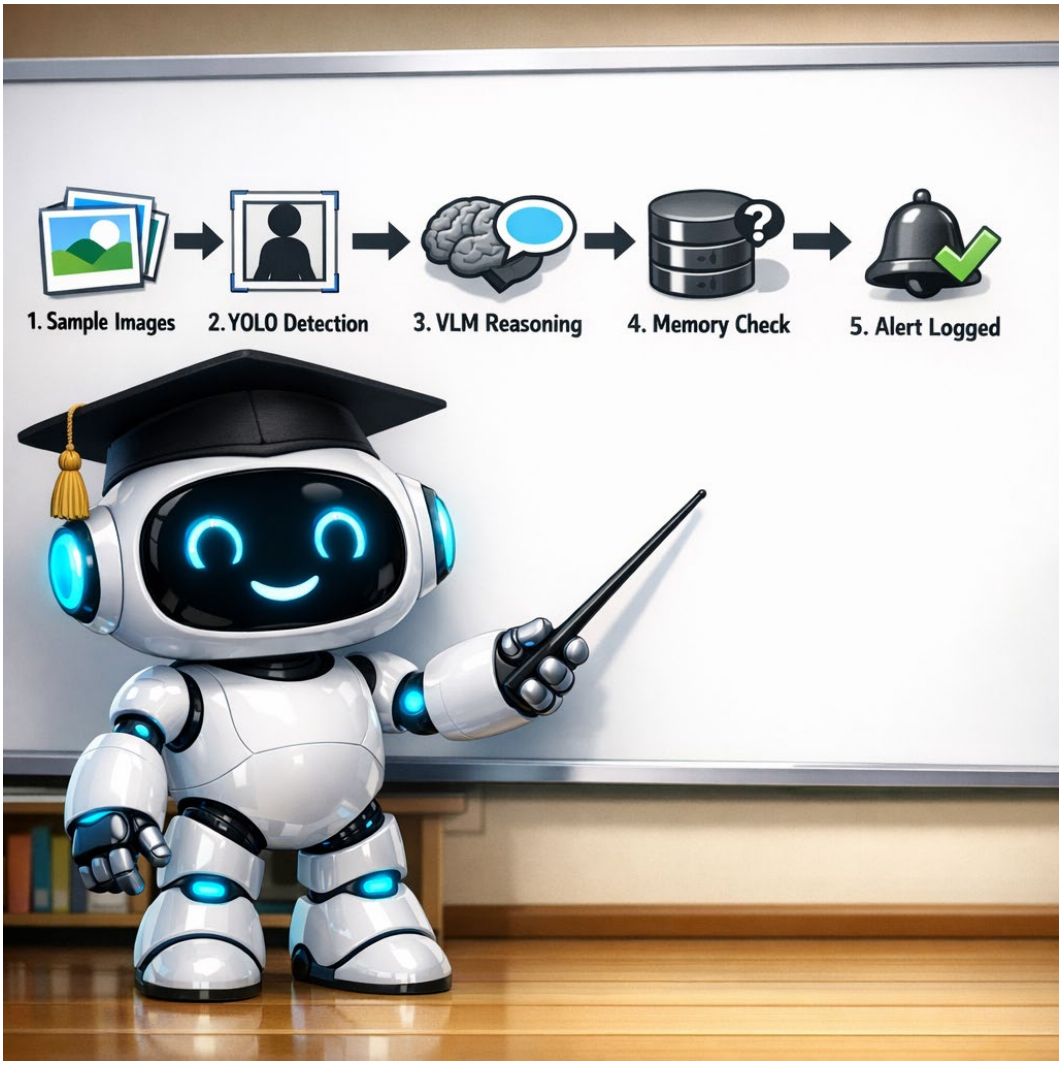



*Figure 21. The revised lab includes PPE-specific detection, constrained VLM review, memory, and a human-review queue.*

## What You Will Not Build

You will not train a model from scratch, deploy the system to a live workplace, or allow it to take disciplinary or safety-critical actions. The lab is calibrated to teach system wiring, evaluation, and responsible review using pretrained models and controlled API calls.

## What to Submit

Submit a short report of approximately two pages. Include screenshots from three or four sample images, a small results table, at least one false positive or false negative, a paragraph about strengths, a paragraph about limitations and privacy, and a reflection describing what you would improve with another week. The full grading rubric is in the lab notebook.

**INDUSTRY PRACTICE | The lab is portfolio material**

A well-executed Module 09 lab can become portfolio material when it is presented responsibly. Record a short demonstration, explain the architecture, show evaluation results, disclose limitations, and describe the human-review step. Employers value candidates who can discuss failure modes as clearly as successful outputs.

## 20. Looking Ahead to Module 10

Module 09 has focused primarily on image-based decisions and short sequences. Module 10 extends the perception layer to video and asks what changes when objects and events unfold over time.

### Why Video Is Harder

A single image is a snapshot. A video is a sequence. The agent has to keep track of objects across frames, handle objects that leave and return, and reason about motion. None of those problems exist in single image work. All of them appear in video. The technical name for keeping track of objects across frames is object tracking, and it is the main subject of Module 10.

### What Tracking Adds

Tracking gives each detected object a stable identity across time. Person number seven walked into the frame, then partially out of view behind a forklift, then back into view. Without tracking, the agent sees three separate detections of unrelated people. With tracking, the agent knows it is the same person and can reason about their trajectory.

Tracking also supports measurements that single frame work cannot. How fast was the person moving? How long have they been in the restricted zone? Did the truck slow down or speed up before it crossed the line? All of these require tracking, which means all of them require Module 10.

## How Module 09 and Module 10 Fit Together

Most real CV agents need both single frame work and tracking. The Module 09 perception step gives you detection, segmentation, and visual reasoning on a single frame. The Module 10 tracking step gives you identity and motion across frames. Together they form the perception layer of a real-time CV agent.


*Figure 22. Module 10 adds identity and motion across frames to the image-based systems introduced in Module 09.*

**KEY TAKEAWAY | Where the course is heading**

Module 08 introduced modern visual models. Module 09 connected models, tools, memory, evaluation, and actions into vision-capable systems. Module 10 adds temporal reasoning through tracking, and Module 11 addresses deployment, optimization, and final review.

**THINK ABOUT IT | Final summary practice**

If you had to teach Module 09 to a friend in one paragraph, with no slides or notes, what would you say? Try it out loud or on paper. The exercise of compressing a module into one paragraph is what makes the ideas stick. Future you, in a job interview, will thank present you for the practice.

# References and Further Reading

Where to go deeper if Module 09 sparked your curiosity

## Foundational Books

**Russell, S. and Norvig, P. (2021).** Artificial Intelligence: A Modern Approach. Fourth edition. Pearson. The standard graduate level reference for AI in general and agents in particular. Chapter 2 introduces the formal definition of an agent that all later work builds on.

**Goodfellow, I., Bengio, Y., and Courville, A. (2016).** Deep Learning. MIT Press. The canonical textbook for the underlying models. Useful for understanding what is inside the building blocks before you wire them into agents.

**Sutton, R. and Barto, A. (2018).** Reinforcement Learning: An Introduction. Second edition. MIT Press. The standard reference for the older lineage of agent research. Optional for ITAI 1378 but essential if you continue to ITAI 2376.

## Recent Books, 2024 to 2026

**Torralba, A., Isola, P., and Freeman, W.T. (2024).** Foundations of Computer Vision. MIT Press. The most current general-purpose CV textbook, written by three leading researchers at MIT. Strong on the foundation models and transformer-based methods that underpin modern CV agents.

**Huyen, C. (2025).** AI Engineering: Building Applications with Foundation Models. O'Reilly. The single most useful applied reference for working with VLMs and agents in production. Strong on evaluation, prompt design, and real deployment tradeoffs.

**Albada, M. (2025).** Building Applications with AI Agents. O'Reilly. Practical guide to designing and shipping agentic systems, with chapters on LangGraph, AutoGen, CrewAI, and the OpenAI Agents SDK. A natural next step after Module 09.

**Lanham, M. (2025).** AI Agents in Action. Manning. Hands-on tour of agent architectures including tool-using agents, multi-agent systems, and computer vision components. Aimed at intermediate Python programmers.

**Alammar, J. and Grootendorst, M. (2024).** Hands-On Large Language Models. O'Reilly. Foundational understanding of the language models that sit at the core of agent reasoning. Useful background even if your focus is the CV side.

## Foundational Papers

**Yao, S., et al. (2022).** ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629, published at ICLR 2023. The paper that introduced the ReAct pattern. Worth reading in full.

**Wei, J., et al. (2022).** Chain of Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS. The paper that established prompted reasoning as a usable technique. Background for understanding how the reasoning step in a modern agent works.

**Shinn, N., et al. (2023).** Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS. A widely cited variation on ReAct that adds a self critique step.

**Kirillov, A., et al. (2023).** Segment Anything. ICCV. The SAM paper. Worth knowing because SAM is one of the most common tools a CV agent calls.

**Liu, H., et al. (2023).** Visual Instruction Tuning. NeurIPS. The LLaVA paper. Background for how modern VLMs are built and why they can serve as the reasoning brain of a CV agent.

## Documentation and Tutorials

**Anthropic.** Computer Use Guide and API Reference. The official documentation for the Computer Use feature. Read this before you try to use the tool in any production setting.

OpenAI. ChatGPT Agent documentation and system card. Official resources describing visual and text-based browsing, tools, user control, and safety considerations.

Microsoft. OmniParser GitHub Repository. Official screen-parsing implementation for converting user-interface screenshots into structured, grounded elements for an outer agent.

ByteDance. UI-TARS and Agent TARS repositories and model documentation. Official resources for GUI-agent models and visual computer-use tooling.

**Hugging Face.** Transformers Library and Agents Documentation. The single most useful resource for actually building a CV agent from code. Free, well organized, and updated continuously.

**LangChain.** Official Documentation and Cookbook. The starting point if you decide to use LangChain in your final project.

Model Context Protocol. Official specification and documentation for the open standard that connects AI applications with tools, resources, and external systems.

NIST. Artificial Intelligence Risk Management Framework and Generative AI Profile. Practical guidance for identifying, measuring, and managing AI risks.

Ultralytics. Construction-PPE dataset documentation. Reference for PPE-specific object classes and model training or evaluation.

## Closing Note

**HCC AI and Robotics Program.** All course materials, slide decks, and lab notebooks for ITAI 1378 are maintained internally and updated each semester. If a model name or product in this booklet is out of date by the time you read it, that is expected. The categories should still hold. The references should still be useful. Your instructor and the program coordinator are happy to help you find the current best version of any tool referenced here.